

```
from google.colab import files
uploaded = files.upload() # This will prompt you to upload the file
```

Choose Files bearing_fail...taset (1).csv

- bearing_failure_dataset (1).csv(text/csv) - 158164 bytes, last modified: 4/29/2025 - 100% done

Saving bearing_failure_dataset (1).csv to bearing_failure_dataset (1) (1).csv

```
import pandas as pd
import io
```

```
# Replace 'bearing_failure_dataset (1) (1).csv' with the actual uploaded filename
df = pd.read_csv(io.BytesIO(uploaded['bearing_failure_dataset (1) (1).csv']))
print(df.head()) # Display the 10 rows
```

```

  ID  Temperature (°C)  Vibration (mm/s)  Speed (RPM)  Lubricant Type \
0   1      97.450712      4.399355      1597.446518      Mineral
1   2      87.926035      3.924634      1756.644399      Mineral
2   3      99.715328      3.059630      1562.274024      Mineral
3   4      112.845448      2.353063      1707.611541      Synthetic
4   5       86.487699      3.698223      1231.915600      Semi-Synthetic

  Load (kN)  Operating Time (hrs)  Material  Shaft Misalignment (mm) \
0   2.959545      1773.498722      Steel      0.076676
1   3.815762      1342.361204      Composite      0.071255
2   2.245914      1227.861721      Ceramic      0.063413
3   2.904894       786.762778      Steel      0.084610
4   5.074185       967.726166      Ceramic      0.078694

  Maintenance Frequency (per month)  Fault Frequency (per 1000 hrs)  Failure
0                                   3                        0.509082      No
1                                   1                        0.071799      No
2                                   2                        0.687050      No
3                                   3                        0.578112      No
4                                   3                        0.659600      No
```

```
x=df[['Vibration (mm/s)']]
y=df['Fault Frequency (per 1000 hrs)']
import numpy as np
from sklearn.linear_model import LinearRegression
o=LinearRegression()
o.fit(x,y)
o.predict([[2]])
```

```

/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with featu
warnings.warn(
array([0.50409102])
```

```
o.score(x,y)
```

```
0.00023753949642946193
```

```
x=df[['Vibration (mm/s)']]
y=df['Fault Frequency (per 1000 hrs)']
import numpy as np
from sklearn.linear_model import Ridge
o=Ridge()
o.fit(x,y)
o.predict([[3]])
```

```

/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Ridge was fitted with feature names
warnings.warn(
array([0.49960174])
```

```
o.score(x,y)
```

```
0.19413033690992587
```

```
x=df[['Vibration (mm/s)']]
y=df['Fault Frequency (per 1000 hrs)']
import numpy as np
from sklearn.tree import DecisionTreeRegressor
o= DecisionTreeRegressor()
o.fit(x,y)
o.predict([[3]])
```

```

/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but DecisionTreeRegressor was fitted with f
warnings.warn(
array([0.4258926])
```

```
o.score(x,y)
```

```
0.0002375392500780782
```

```
x=df[['Vibration (mm/s)']]
y=df['Fault Frequency (per 1000 hrs)']
import numpy as np
from sklearn.tree import ExtraTreeRegressor
o= ExtraTreeRegressor()
o.fit(x,y)
o.predict([[3]])
```

```

/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but ExtraTreeRegressor was fitted with feat
warnings.warn(
array([0.4258926])
```

```
o.score(x,y)
```

```
0.9933058531671696
```

```
x=df[['Vibration (mm/s)']]
y=df['Fault Frequency (per 1000 hrs)']
import numpy as np
from sklearn.ensemble import RandomForestRegressor
o= RandomForestRegressor()
o.fit(x,y)
o.predict([[3]])
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but RandomForestRegressor was fitted with 1
warnings.warn(
array([0.52263496])
```

```
o.score(x,y)
```

```
0.7808901490587472
```

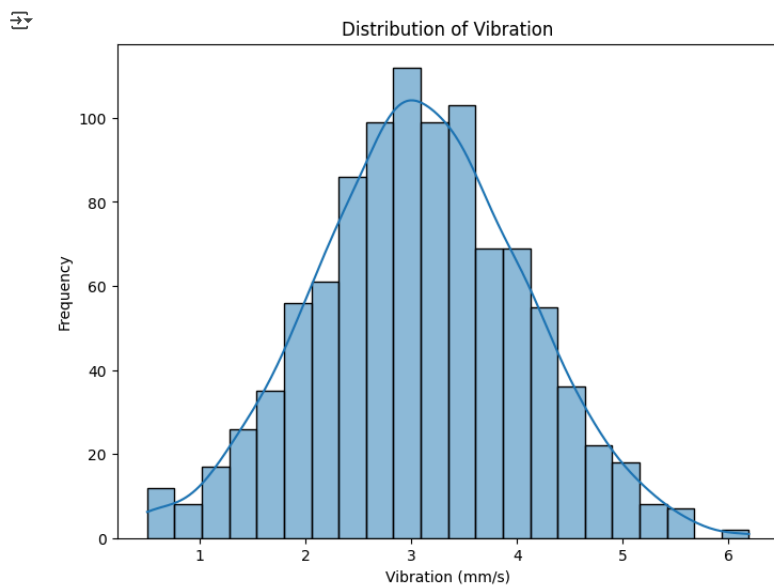
```
x=df[['Vibration (mm/s)']]
y=df['Fault Frequency (per 1000 hrs)']
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor
o= GradientBoostingRegressor()
o.fit(x,y)
o.predict([[3]])
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but GradientBoostingRegressor was fitted w
warnings.warn(
array([0.50817289])
```

```
o.score(x,y)
```

```
0.19413033690992587
```

```
import matplotlib.pyplot as plt
import seaborn as sns
# Chart 1: Histogram of Vibration
plt.figure(figsize=(8, 6))
sns.histplot(df['Vibration (mm/s)'], kde=True) # KDE adds a smooth curve
plt.title('Distribution of Vibration')
plt.xlabel('Vibration (mm/s)')
plt.ylabel('Frequency')
plt.show()
```



Generate 10 random numbers using numpy

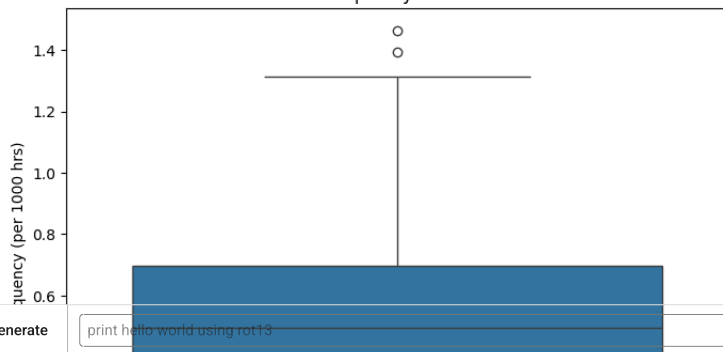


Close

```
import matplotlib.pyplot as plt
import seaborn as sns
# Chart 2: Boxplot of Fault Frequency
plt.figure(figsize=(8, 6))
sns.boxplot(y='Fault Frequency (per 1000 hrs)', data=df) #or x = 'Fault Frequency...' for horizontal boxplot
plt.title('Fault Frequency Distribution')
plt.ylabel('Fault Frequency (per 1000 hrs)')
plt.show()
```



Fault Frequency Distribution



Generate

print hello world using rot13

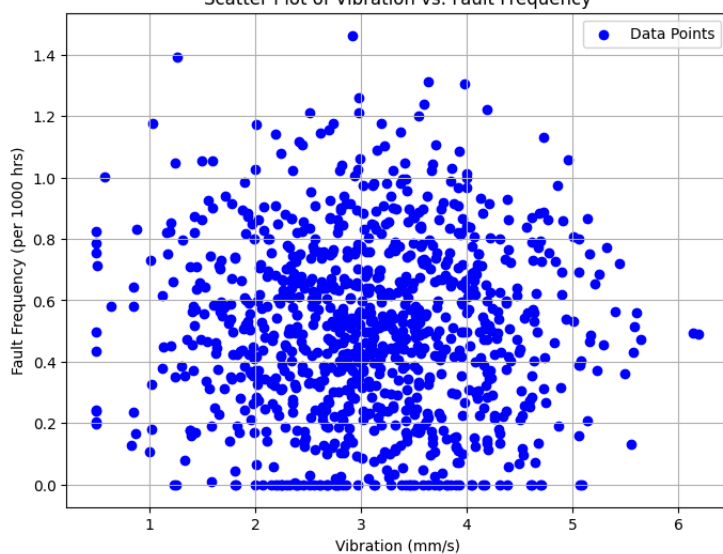


Close

```
import numpy as np
import matplotlib.pyplot as plt
# 3. Scatter Plot
plt.figure(figsize=(8, 6))
plt.scatter(df['Vibration (mm/s)'], df['Fault Frequency (per 1000 hrs)'], color='blue', label='Data Points')
plt.xlabel('Vibration (mm/s)')
plt.ylabel('Fault Frequency (per 1000 hrs)')
plt.title('Scatter Plot of Vibration vs. Fault Frequency')
plt.legend()
plt.grid(True)
plt.show()
```



Scatter Plot of Vibration vs. Fault Frequency



Generate

a slider using jupyter widgets



Close

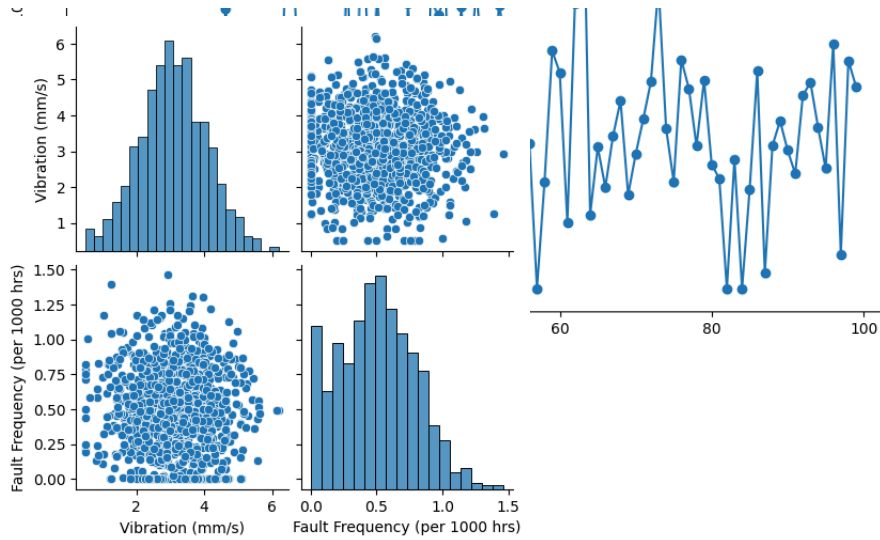
```
import numpy as np
import matplotlib.pyplot as plt
# 4. Line Plot (example with a subset of data - replace with your desired x-axis)
plt.figure(figsize=(10, 6))
plt.plot(df.index[:100], df['Fault Frequency (per 1000 hrs)'][:100], marker='o', linestyle='--') # Example: first 100 data points
plt.title('Fault Frequency Over Time (Example)') # Change title accordingly
plt.xlabel('Index/Time') # Change label if needed
plt.ylabel('Fault Frequency (per 1000 hrs)')
plt.show()
```



Fault Frequency Over Time (Example)

```
import numpy as np
import matplotlib.pyplot as plt
# 5. Pairplot (if you have more numerical columns, this is useful)

# Example showing only two variables to keep the chart readable
sns.pairplot(df[['Vibration (mm/s)', 'Fault Frequency (per 1000 hrs)']])
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt
# 4. Violin Plot
plt.figure(figsize=(8, 6))
sns.violinplot(y='Fault Frequency (per 1000 hrs)', data=df)
plt.title('Distribution of Fault Frequency')
plt.ylabel('Fault Frequency (per 1000 hrs)')
plt.show()
```



Distribution of Fault Frequency

